CDC _ RDC

BY: Eng Ali Eltemsah

© 2022 Eng. Ali Eltemsah

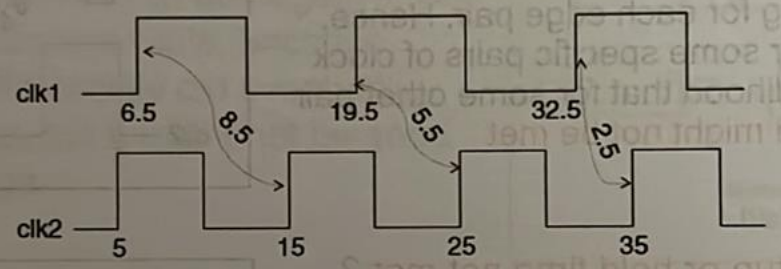
CONTENTS

- Introduction
- CDC Problems
  - Meta-stability
  - Data Incoherence
  - Data Loss
- Synchronization techniques
  - Multi-Flop Synchronization Scheme
  - Multi-Flop/Gray code Synchronization Scheme
  - Synchronized Enable Synchronization Scheme
  - Synchronized MUX-Select Synchronization Scheme
  - FIFO Synchronization Scheme
  - Ready-acknowledge handshaking
- FIFO Depth Calculation.
- Reset Synchronizer

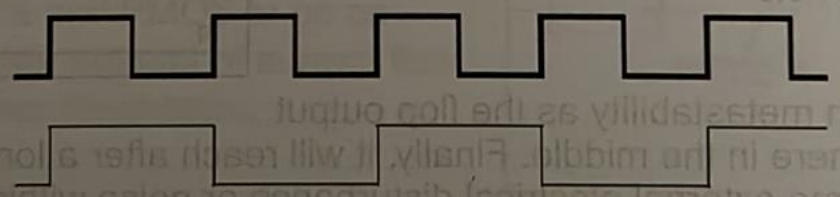
© 2022 Eng. Ali Eltemsah

Introduction

- Two clocks are considered to be in **two different domains** or **asynchronous to each other**:-
  - if the phase relationship between their active edges can not be determined predictably or the time gap between the edges of these two clocks keeps changing continuously, even if they have the same frequency.
  - **Example: different clock source.**

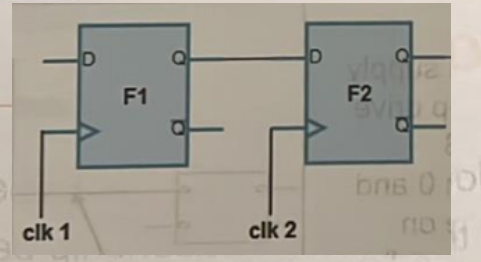


- Two clocks are considered to be in **the same domain** or **synchronous to each other**:-
  - if the phase relationship between the edges can be determined predictably –then the two clocks are considered to be in the same domain even if they have different periods
  - **Example: generated clocks.**



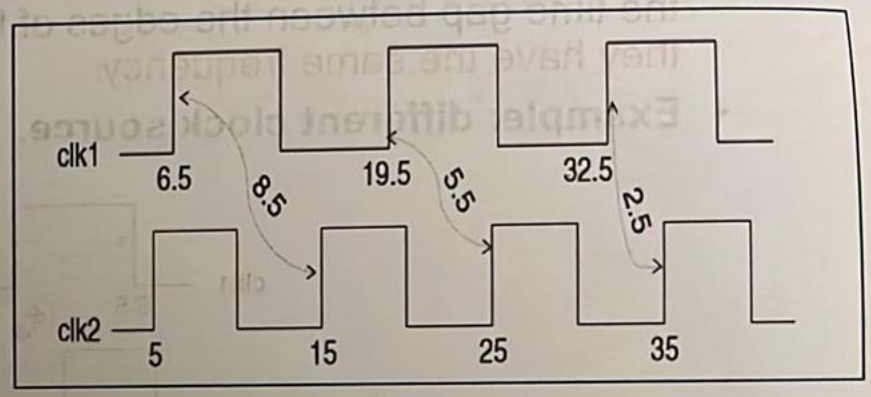
© 2022 Eng. Ali Eltemsah

CDC Problems

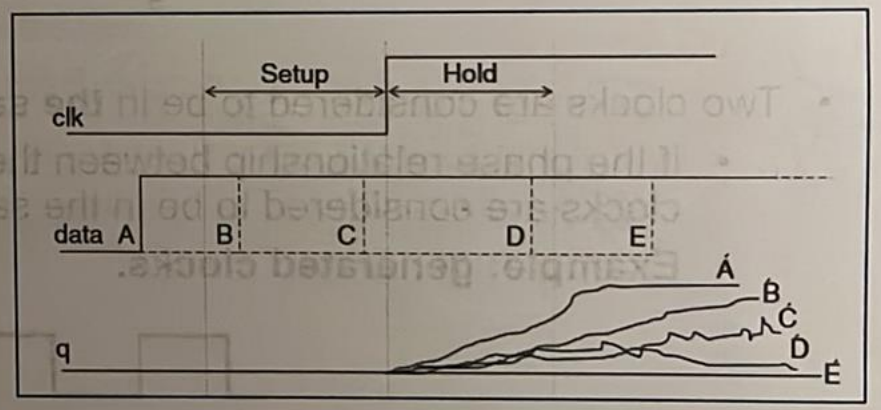


Problem 1: Metastability

- One of the biggest problems due to CDC is metastability as it not guarantee a reliable transfer of data between any pair of registers which setup and hold requirements have to be met. As seen in the figure, the time available for a signal keeps changing for each edge pair. Hence, even if the timing is met for some specific pairs of clock edges, there is quite a likelihood that for some other pair of edges, the setup or hold might not be met



- What happened if the setup or hold time not met ?
  - A meet setup and hold
  - B violate setup
  - C gross violate setup
  - D violate hold
  - E meet setup and hold



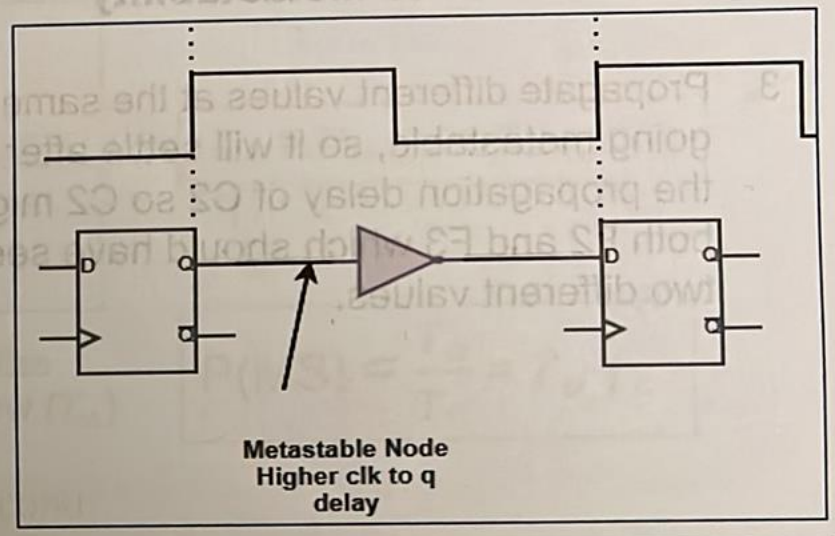
- B, C and D suffer from metastability as the flop output has got stuck somewhere in the middle. Finally, it will reach after a long time somewhere (either a 0 or a 1)– due to some external electrical disturbance or noise within the device.

© 2022 Eng. Ali Eltemsah

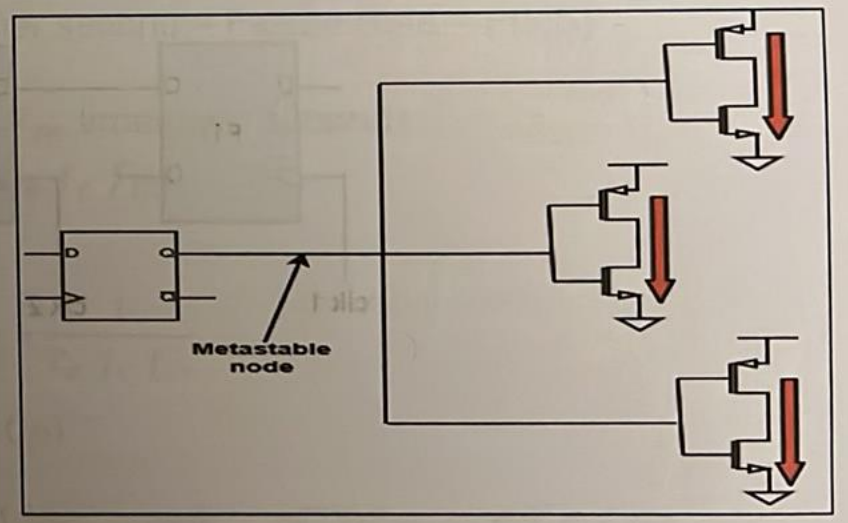
CDC Problems

- **Problems due to Metastability**

1. High clk-to-q delay which play a role in the setup and hold slack for the next flop, so, a higher delay on this flop could cause the setup slack for the next flop to decreased or sometimes to be negative. So, here, the data capture is not reliable, because, though, the flop has metastability did eventually reach the intended value, but it might not be good enough for the next stage.



2. Establishing of a short circuit path between supply and ground of a CMOS transistors if the flop drive a combinational logic as the input of CMOS transistors is somewhere mid-way between 0 and 1 which allow both NMOS and PMOS to be on simultaneously, so a higher amount of current flow, which means higher power dissipation.

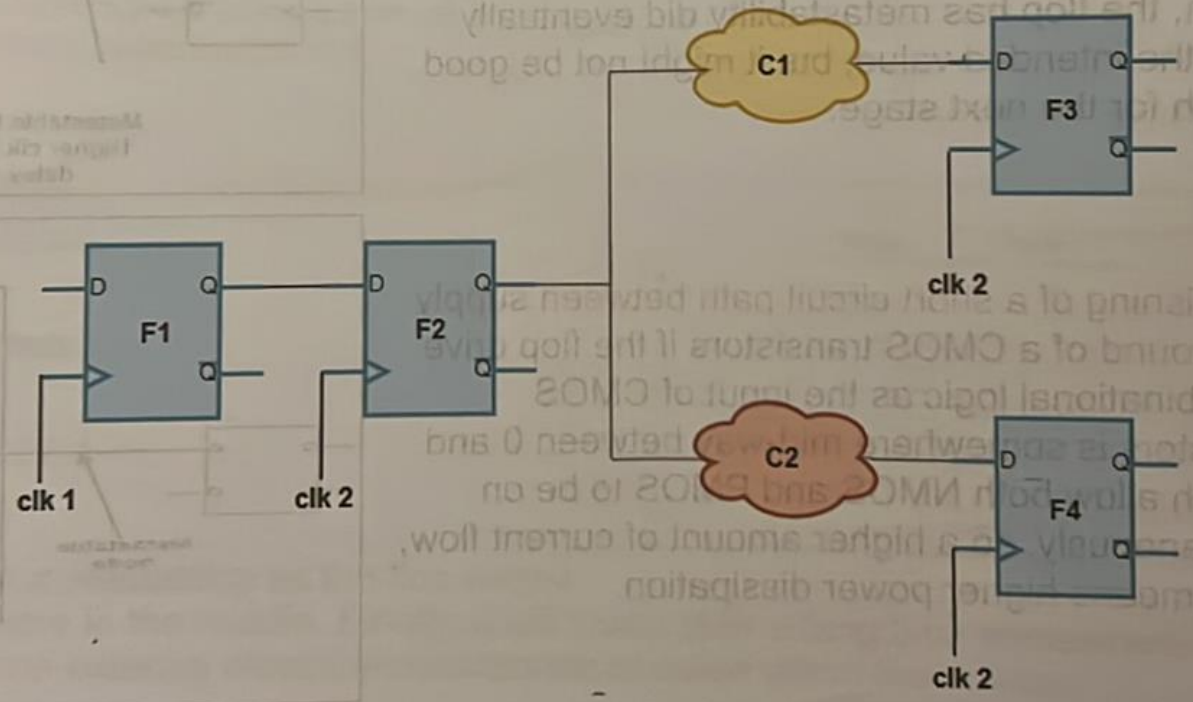


© 2022 Eng. Ali Eltemsah

CDC Problems

- **Problems due to Metastability**

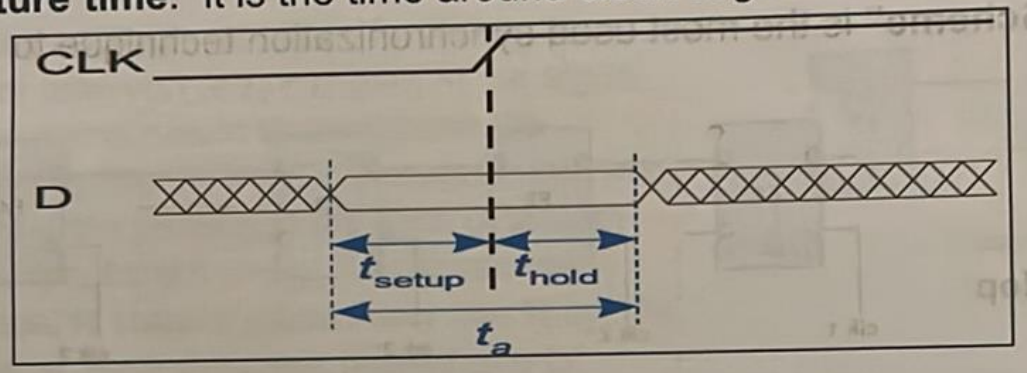
3. Propagate different values at the same time due to clock crossing and possibility of F2's output going metastable, so it will settle after longer time so if propagation delay of C1 is greater than the propagation delay of C2 so C2 might capture the settled value while C1 not So, effectively, both F2 and F3 which should have seen the same value at F2's outputs are actually seeing two different values.



© 2022 Eng. Ali Eltemsah

Probability of Metastability

- **Aperture time**: it is the time around clock edge data must be stable



$$T_a = T_{setup} + T_{hold}$$

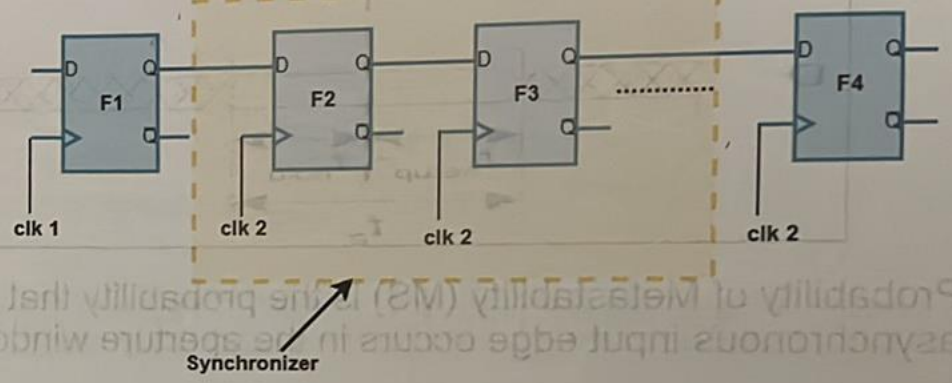
- Probability of Metastability (MS) is the probability that the asynchronous input edge occurs in the aperture window (T_a)

$$P(MS) = \frac{T_a}{T_C} = T_a f_c$$

- Assume the asynchronous input changes once per second
  - Probability of MS per second = probability of failure per second = Failure Rate = P(MS)
- If the asynchronous input changes at a rate of f_{in} (i.e.,f_{in} times per second)
  - Probability of failure per second = Failure Rate (FR) = $T_a f_c f_{in}$
- Failure happens (on average) every $1/FR$
  - Called mean time between failures (MTBF) = MTBF = $\frac{1}{FR} = \frac{1}{T_a f_c f_{in}}$
- Example : (Assume $f_c = 1GHz$, $f_{in} = 100MHz$, $T_a = 10p$)
  - $P(MS) = f_c T_a = 0.01$ ➡ $FR = f_{in} f_c T_a = 1$ M/s ➡ MTBF = 1/ FR = 1 us

Bit Synchronization Techniques

- **"Multi-Flop Synchronization Scheme"** is the most used synchronization technique to synchronize **single bit**.
  - **F2 is called destination flop**
  - **F3 is called synchronization flop**

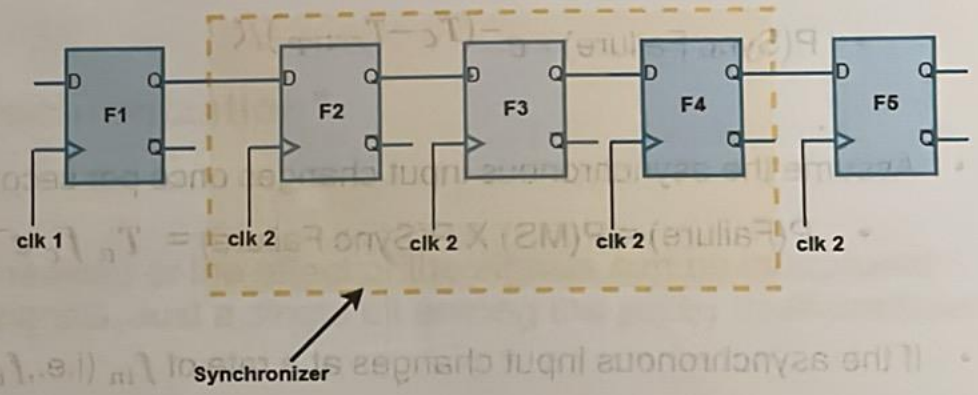


- In double FF synchronizer, the first flip-flop samples the asynchronous input signal into the destination clock domain and waits for a full destination clock cycle to permit any meta-stability on the stage-1 output signal to decay, then the stage-1 signal is sampled by the same clock into a second stage flip-flop, with the intended goal that the stage-2 signal is now stable and valid synchronized signal into the destination clock domain.
- You should not put any logic between F2 and F3. By putting any such logic, you effectively reduce the time given to F1 to settle to be captured by F3. And also, the gates forming this logic could see high short circuit, during the duration that F1's output is not settled.
- It is theoretically possible with very low probability that stage-1 signal after full destination clock cycle is still meta-stable, by the time this signal is clocked into the second stage will cause the stage-2 signal to also go meta-stable, so in this case we need to add a third stage to reduce what is called "mean time between synchronization failures (MTBF)"

© 2022 Eng. Ali Eltemsah

Bit Synchronization Techniques

- When adding more flops in series, the more time you are providing to the signal to be come stable that increase the reliability factor "MTBF" However, you are paying the penalty in the form of latency – your results get delayed by those many cycles. N stage synchronizer has N cycles latency. So a designer might have to compromise between latency and MTBF

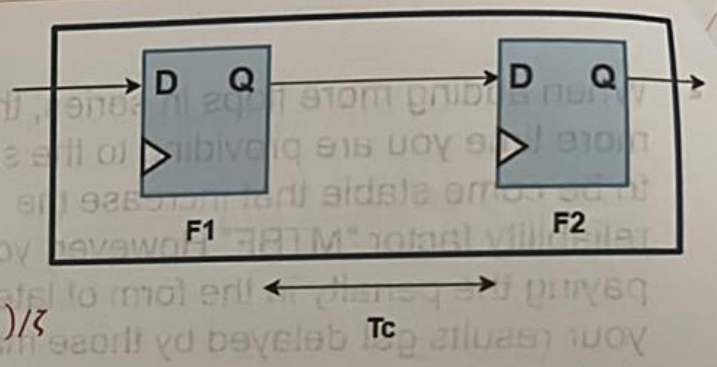


- **Notes:-**
  - Sometimes, your cell library might provide for special flops –that have a tendency to come out of metastable state much faster. Such flops are called **meta-hardened flops.** This reduces the chances that synchronization flop to become metastable. So, using meta-hardened flops increase the MTBF significantly.
  - MBTF is inversely proportional with the speed clocks, so the two stages flop synchronizers are not enough for very high speed clocks as MTBF becomes significantly low at higher frequencies
  - Multi-flop sync cells creates **N clock cycles of latency**. The double FF synchronizer has latency 2 clock cycles, one of methods used to decrease the latency to only 1 clock cycle is to use a negative edge destination flop and positive edge synchronization flop, but the synchronization process becomes a little bit worse.

"Meta-stability can NEVER be eliminated!!!!! Can only be isolated to increase MTBF"

© 2022 Eng. Ali Eltemsah

Probability of Synchronizer Failure



- Probability of synchronizer failure
  - P(Sync Failure) = $e^{-(T_C - T_{setup})/\zeta}$
- Assume the asynchronous input changes once per second
  - P(Failure) = P(MS) X P(Sync Failure) = $T_a\, f_c\, e^{-(T_C - T_{setup})/\zeta}$
- If the asynchronous input changes at a rate of f_{in} (i.e., f_{in} times per second)
  - FR (Failure Rate) = P(MS) X P(Sync Failure) X Input Rate = $T_a\, f_c\, f_{in}\, e^{-(T_C - T_{setup})/\zeta}$
- MTBF = $\frac{1}{FR} = \frac{1}{T_a\, f_c\, f_{in}\, e^{-(T_C - T_{setup})/\zeta}} = \frac{e^{+(T_C - T_{setup})/\zeta}}{T_a\, f_c\, f_{in}}$
- For N FF synchronizer ➡ MTBF = $\frac{1}{FR} = \frac{e^{+N(T_C - T_{setup})/2\zeta}}{T_a\, f_c\, f_{in}}$
- Factors to increase MTBF ➡
  1) decrease input change rate
  2) decrease clock frequency
  3) increase Number of FF

© 2022 Eng. Ali Eltemsah

CDC Problems

Problem 2: Data Incoherence

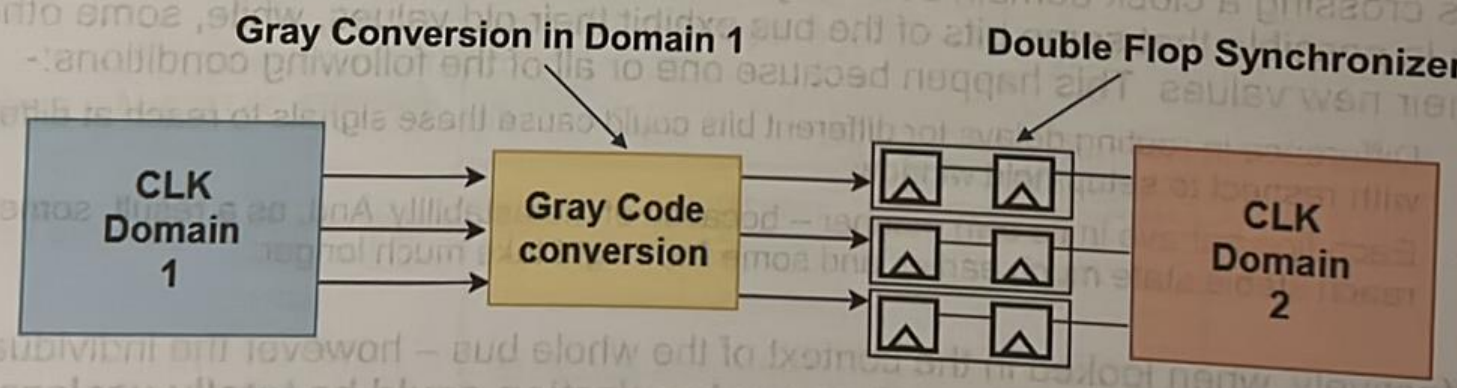
- It is one of the challenges with **"Bus synchronization".**
- **Bus Synchronization**
  - **Bus means a set of signals**, where the meaning or the effect of the signals can be interpreted only in the context of the complete set of signals. Just a single bit among the set by itself does not mean anything. Examples could include:
    - Vectored Data Lines
    - Address Lines
    - Control Bus
- If a bus is crossing a clock domain boundary and each signal is synchronized individually. In such a case ,it is possible that some bits of the bus exhibit their old values, while, some other bits exhibit their new values. This happen because one or all of the following conditions:-
    - Difference in routing delays for different bits could cause these signals to reach at different points with respect to setup/hold window.
    - Each flop behave in its own manner – because of metastability. And, as a result, some flop might reach stable state much earlier, and some flop might take much longer.
- Thus, effectively, when looked in the context of the whole bus – however the individual bits are synchronized, **the data on the bus after synchronization could be totally useless**

© 2022 Eng. Ali Eltemsah

Bus Synchronization Techniques

1. Gray Coding Technique / Multi Flop Synchronizers Method

- One of the solutions is using Gray encoded signals mean that only a single bit changes. Thus, for all other bits, whether the individual synchronizer captures the new data or the old data, it does not really matter, because either way, it's the same value. Only for one bit (that changed).
- The new data or the old value is different. However, irrespective of which data is captured for this one bit, the final value of the entire bus is still meaningful and not garbage.
- Gray encoding can be used only if the consecutive values have to follow a pattern. If the consecutive values is random and do not follow any pattern, then, grey encoding is also not possible.

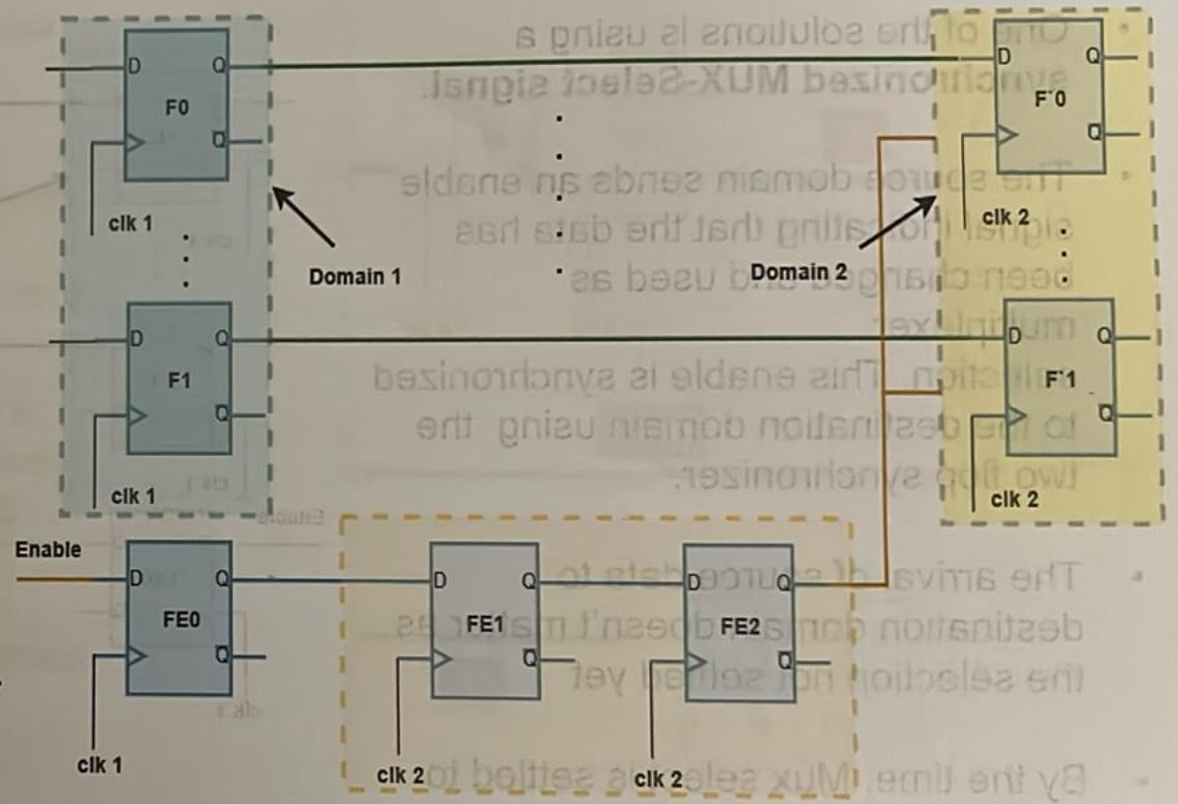


© 2022 Eng. Ali Eltemsah

Bus Synchronization Techniques

2. Enable Based Synchronization Method

- One of the solutions is using a **synchronized enable signal**.
- The source domain sends an enable signal indicating that the data has been changed. This enable is synchronized to the destination domain using two flop synchronizer. The arrival of source data to destination domain doesn't matter.
- All clocks on destination domain will be ignored until the valid enable is arrived. Hence, even if there is a setup/hold violation it does not matter.
- By the time, enable signal passes through the synchronization flops FE1 and FE2 until it settles.



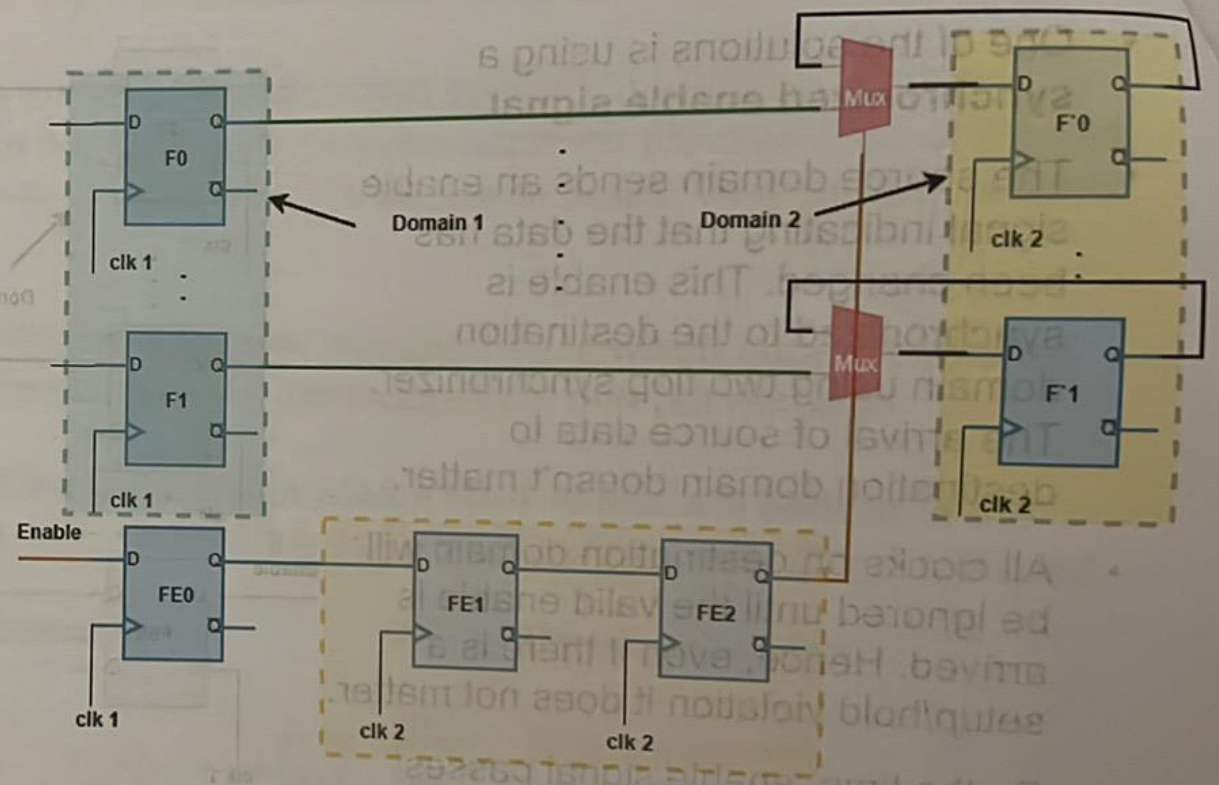
- It is expected that after the duration of settling the enable signal, the individual data bits would have also settled. So, the entire bus can now be captured into the destination flops. So the main responsibility of enable signal is to guarantee that the data bits is settled by giving it more time.

© 2022 Eng. Ali Eltemsah

Bus Synchronization Techniques

3. Synchronized MUX-Select Synchronization Method

- One of the solutions is using a **synchronized MUX-Select signal**.
- The source domain sends an enable signal indicating that the data has been changed and used as multiplexer selection. This enable is synchronized to the destination domain using the two flop synchronizer.
- The arrival of source data to destination domain doesn't matter as the selection not settled yet
- By the time, Mux select is settled to pass the data coming from domain 1 which take enough time to be settled to be captured by domain 2 flops

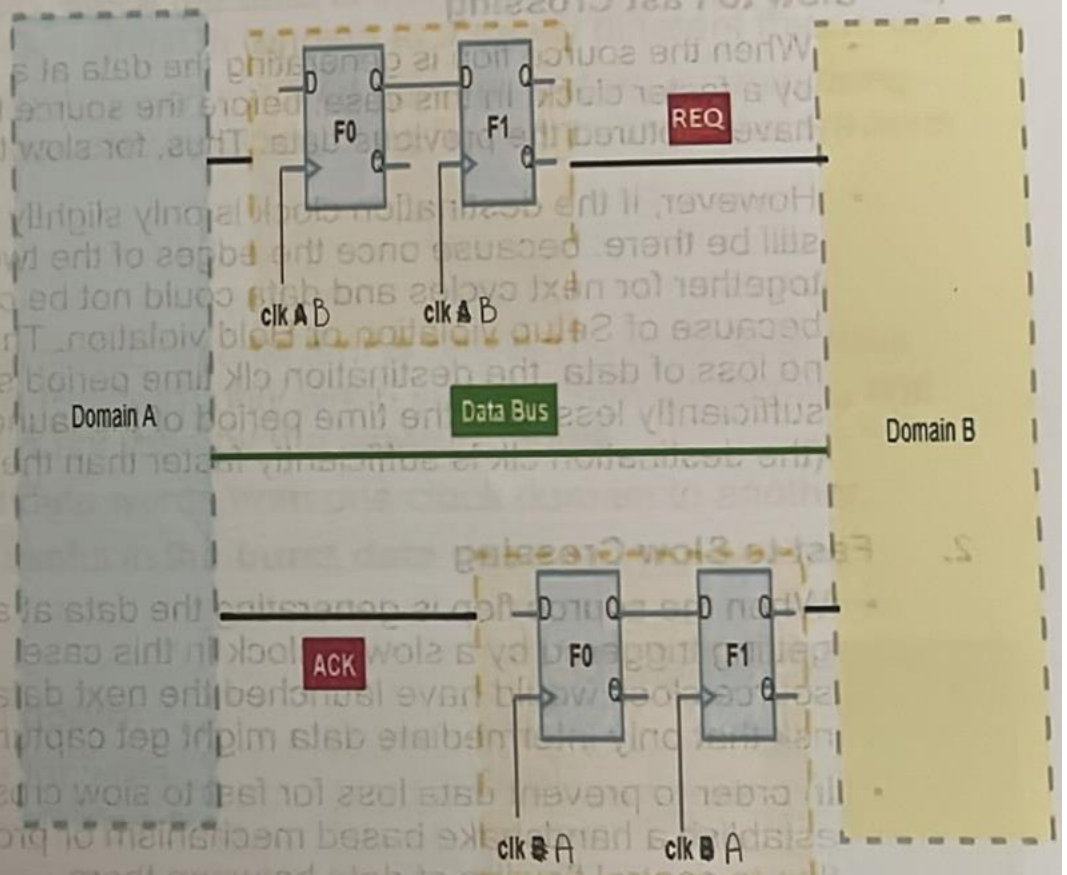


© 2022 Eng. Ali Eltemsah

Bus Synchronization Techniques

4. Ready-Acknowledge handshaking Synchronization Method

- One of the solutions is using simple **two-line (Ready and Acknowledge) handshaking Sequence**.
- The sender **"Domain B"** places data onto a data bus and then synchronizes a request signal (**REQ**) to the receiving clock domain "**Domain A**".
- When the "**REQ** " signal is recognized in the destination clock domain, the receiver clocks the data into a register (the data should have been stable for at least two/three sampling clock edges in the destination clock domain) and then passes an acknowledgement signal **"ACK"** through a synchronizer to the sender.
- When the sender recognizes the synchronized **"ACK"** signal, the sender can change the value being driven onto the data bus and release the **REQ** signal

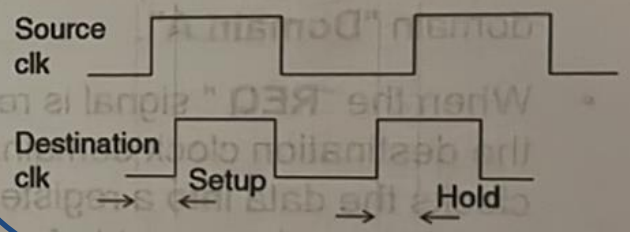


© 2022 Eng. Ali Eltemsah

CDC Problems -> Data Loss

- There are two types of crossing:-

1. **Slow to Fast Crossing**
   - When the source flop is generating the data at a lower frequency. And, the destination flop is triggered by a faster clock. In this case, before the source flop generates another data, the destination flop would have captured the previous data. Thus, for slow to fast crossing, there might not be a risk of data loss.
   - However, if the destination clock is only slightly faster than the source clock, the data loss risk would still be there. because once the edges of the two clocks are almost aligned they will come very close together for next cycles and data could not be captured reliably because of Setup violation or Hold violation. Thus, to ensure no loss of data, the destination clk time period should be sufficiently less than the time period of the launch clk (the destination clk is sufficiently faster than the source clk)



2. **Fast to Slow Crossing**
   - When the source flop is generating the data at a higher frequency. And, the destination flop is getting triggered by a slower clock In this case, before the destination flop captures a data, the source clock would have launched the next data. Thus, for fast to slow crossing, there is always a risk that only intermediate data might get captured, and, several data might get lost.
   - In order to prevent data loss for fast to slow crossing between asynchronous clocks, you have to establish a handshake based mechanism or protocol between the launch flop and the destination flop to control flowing of data between them.
   - The handshake mechanism removes the risk of data-loss. However, you pay the cost in terms of additional circuitry (more power and area) for the handshake protocol.

© 2022 Eng. Ali Eltemsah

CDC Problems -> Data Loss -> FIFO

Problem 3: Data Loss

- It is one of the risks associated with CDC, since the data is being generated by one clock, and, is being consumed by another clock – which is running at a totally different frequency.
- It is possible that the launching clock might launch a new data even before the capturing flop was triggered to capture the previous data. If such a thing happens, then, the previous data could be lost.

- **How To Avoid Data Loss ?**
  - **Asynchronous FIFO** (First In First Out) based mechanism is one of the most important techniques to prevent data loss, FIFO is required, only when you are slow at reading and fast in writing to **buffer the data** which is not read by the slower module.
  - FIFOs are used to safely pass multi-bit data words from one clock domain to another.
  - FIFO based mechanism can be found useful in the **burst data case**.

- **FIFO Basics:-**
  - A FIFO is a 2-port memory with certain **depth**.
  - It has two clocks, one for read and one for write.
  - It has two addresses, one for read and one for write.
  - Writing happens at location specified by write address.
  - Reading happens at location specified by read address.

© 2022 Eng. Ali Eltemsah

FIFO -> Depth Calculation

Example 1 : $f_A > f_B$ with no idle cycles in both write and read.

- Writing frequency (f_A)= 80MHz.
- Reading Frequency (f_B) = 50MHz.
- Burst Length = No. of data items to be transferred = 120.
- There are no idle cycles in both reading and writing which means that, all the items in the burst will be written and read in consecutive clock cycles.

Solution :-

1) Time required to **write** one data item = $\frac{1}{80\text{MHz}}$ **= 12.5 nsec**
2) Time required to **read** one data item = $\frac{1}{50\text{MHz}}$ **= 20 nsec**
3) Time required to **write** all the data in the burst = **120 * 12.5 = 1500 nsec**
4) Number of data items can be read in a duration of 1500 **nsec** = $\frac{1500\text{ nsec}}{20\text{ nsec}}$ = 75
5) The remaining no. of bytes to be stored in the FIFO = 120 – 75 = 45.

The minimum depth of the FIFO should be 45.

© 2022 Eng. Ali Eltemsah

FIFO -> Depth Calculation

Example 2 : $f_A > f_B$ with idle cycles in both write and read.

- Writing frequency (f_A)= 80MHz.
- Reading Frequency (f_B) = 50MHz.
- Burst Length = No. of data items to be transferred = 120.
- No. of idle cycles between two successive writes is = 1. (It means that, after writing one data, module A is waiting for one clock cycle, to initiate the next write. So, it can be understood that for every two clock cycles, one data is written)
- No. of idle cycles between two successive reads is = 3. ~~(It means that, after writing one data, module A is waiting for one clock cycle, to initiate the next write. So, it can be understood that for every two clock cycles, one data is written)~~

Solution :-

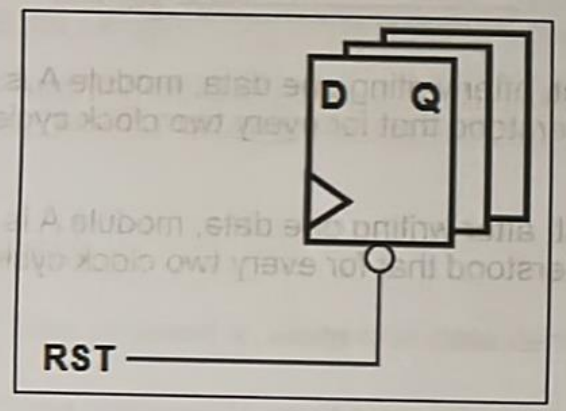
1) Time required to **write** one data item = **2 *** $\frac{1}{80\text{MHz}}$ **= 25 nsec**
2) Time required to **read** one data item = **4 *** $\frac{1}{50\text{MHz}}$ **= 80 nsec**
3) Time required to **write** all the data in the burst = **120 * 25 = 3000 nsec**
4) Number of data items can be read in a duration of 3000 **nsec** = $\frac{3000\text{ nsec}}{80\text{ nsec}}$ = 37.5 >> 37
5) The remaining no. of bytes to be stored in the FIFO = 120 – 37 = 83.

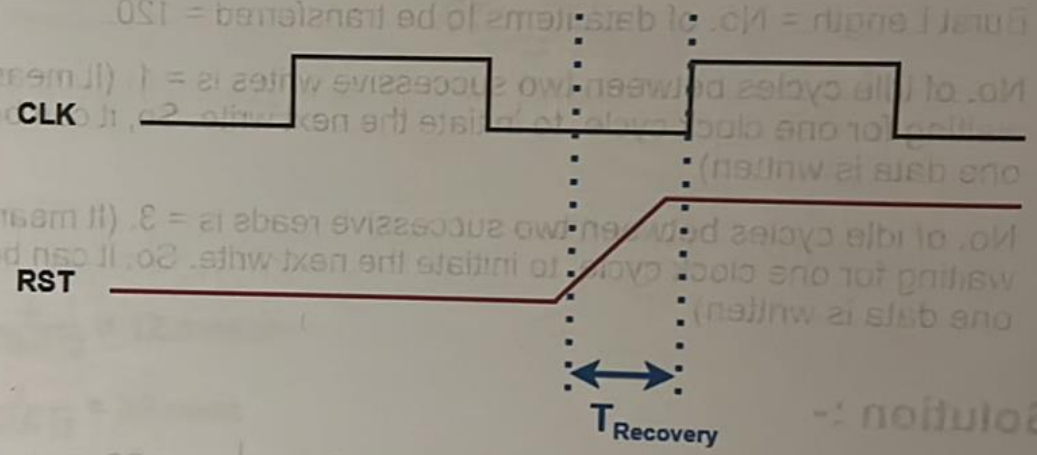
The minimum depth of the FIFO should be 83.

© 2022 Eng. Ali Eltemsah

Reset Synchronizer

- **Why do we need Reset Synchronizer?**
  - We have a problem with the asynchronous reset deassertion as it may violate recovery time which get all the flops into metastable state.



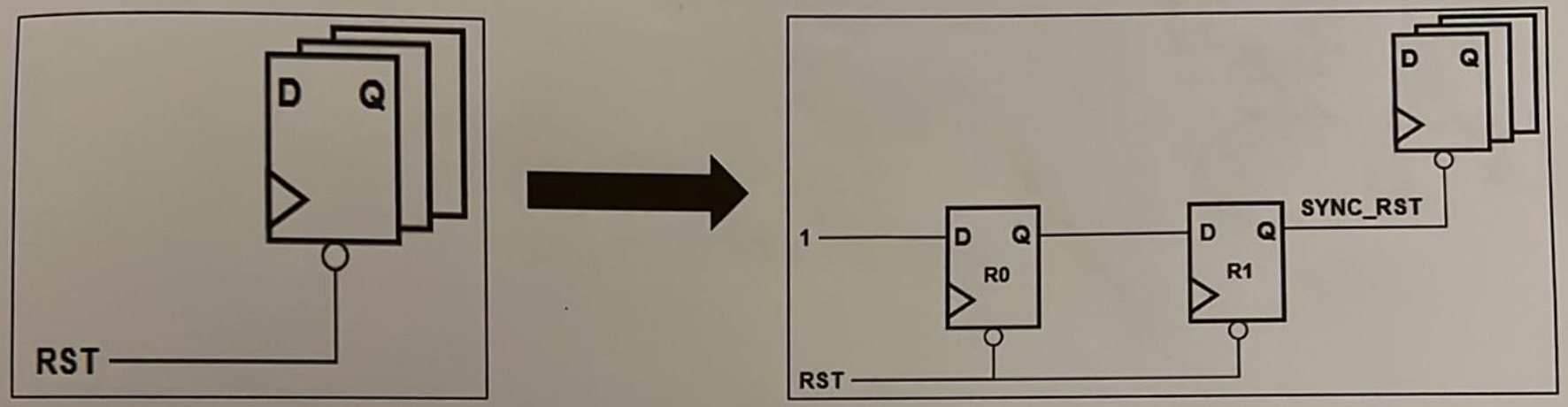


- so we need the designs to have: -
  1. **asynchronous reset assertion:** When reset is asserted, it brings all the flops to reset state whether or not clock is toggling **(assertion should be asynchronous)**
  2. **synchronous reset de-assertion:** When reset is deasserted, wait for a clock edge, and then, deassert the reset **(deassertion should be synchronous)**

- **A reset synchronizer synchronizes the deassertion of reset with respect to the clock domain.**

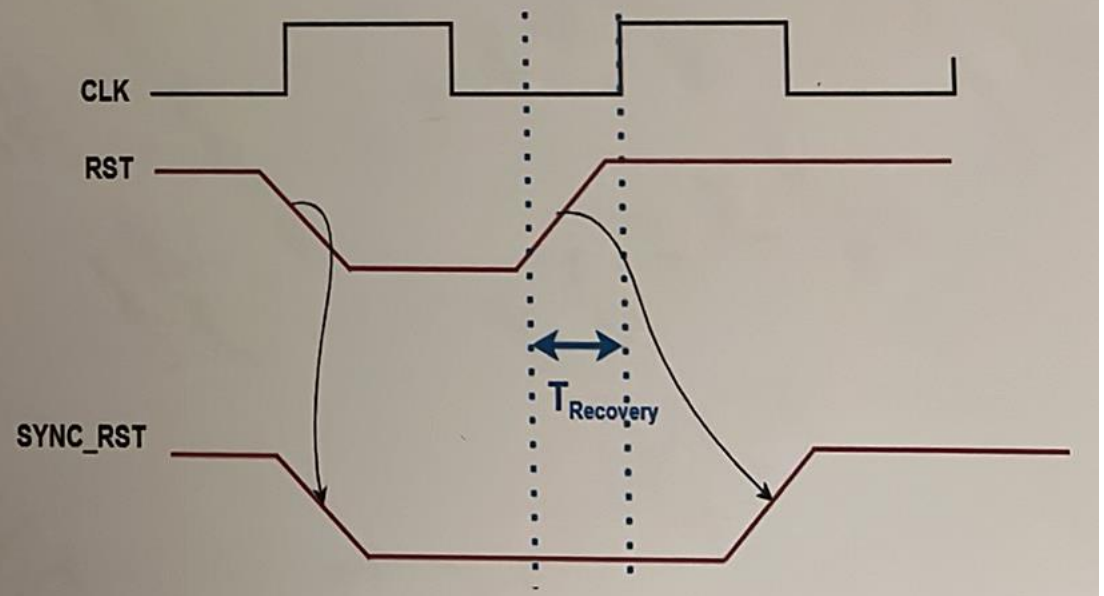
© 2022 Eng. Ali Eltemsah

Reset Synchronizer



Notes: -

- Flop (R0) will suffer from metastability and will settle very to "1" to drive R1
- The Number of reset synchronizers needed equals to the number of clock domains. Each reset synchronizer fans out to all the resettable flops of its own clock domain.



© 2022 Eng. Ali Eltemsah